

Migration des Données Médicales vers MongoDB



Sophana SEAN – 15 décembre 2025

Contexte et Objectifs

- Client confronté à des problèmes de scalabilité avec son dataset médical (55 500 patients).
- Proposition d'une solution Big Data scalable horizontalement : MongoDB (NoSQL).
- Mission : Migration locale sécurisée + Recherches pour déploiement cloud AWS.
- Dataset source : healthcare_dataset.csv (données patients : nom, âge, admission, etc.).

Justification des Choix

MongoDB

NoSQL orienté documents → Flexibilité pour données médicales hétérogènes, scalabilité horizontale native.

Docker

Conteneurisation → Environnement reproductible, portable, sécurisé et facile à déployer.

Python

Scripts migration/CRUD/export (pymongo, pandas) → Simple, puissant pour data processing.

Persistance & Sécurité

Volumes Docker
Authentification par rôles

Déploiement Local avec Docker

`docker-compose.yml`

Services MongoDB + Application Python

Volume persistant **mongo-data**

Pour garder les données entre redémarrages

Initialisation auto

Script JS pour créer users/rôles au premier
lancement

Lancement

`docker compose up --build -d`

Systeme d'Authentication et Rôles

4 utilisateurs créés avec rôles dédiés

Admin

app-user

Admin serveur

admin-db

Application métier
(readWrite)

read-only

Gestion base médicale
(readWrite, dbAdmin,
userAdmin).

Audit/reporting
(lecture uniquement)

Pipeline de Migration

migration.py

Lecture CSV → DataFrame Pandas

Génération patient_id lisible (P00001 à P55500) → Unicité & lisibilité humaine

Nettoyage automatique de la collection

Insertion batch dans collection patients (base medical_db)

Logs détaillés



Pas d'ID unique fiable dans le CSV → Création custom nécessaire

CRUD & Export

crud.py : Create/Read/Update/Delete avec validation (âge, groupe sanguin, dates...)

Index unique sur patient_id : recherche intelligente (par nom/partielle)

export.py : Export JSON/CSV/Excel avec timestamp auto

main.py : Orchestration complète + Démonstration tests (ajout, update, delete, recherche, export)



Structure projet

```
medical-migration/  
├── data/  
│   └── healthcare_dataset.csv  
├── export/  
├── images/  
├── init-db/  
│   └── create-users.js  
├── logs/  
│   └── migration_report.log  
├── PDF/  
├── src/  
│   ├── migration.py  
│   ├── connection.py  
│   ├── crud.py  
│   └── export.py  
├── .env  
├── .gitignore  
├── main.py  
├── docker-compose.yml  
├── Dockerfile  
├── requirements.txt  
└── README.md
```

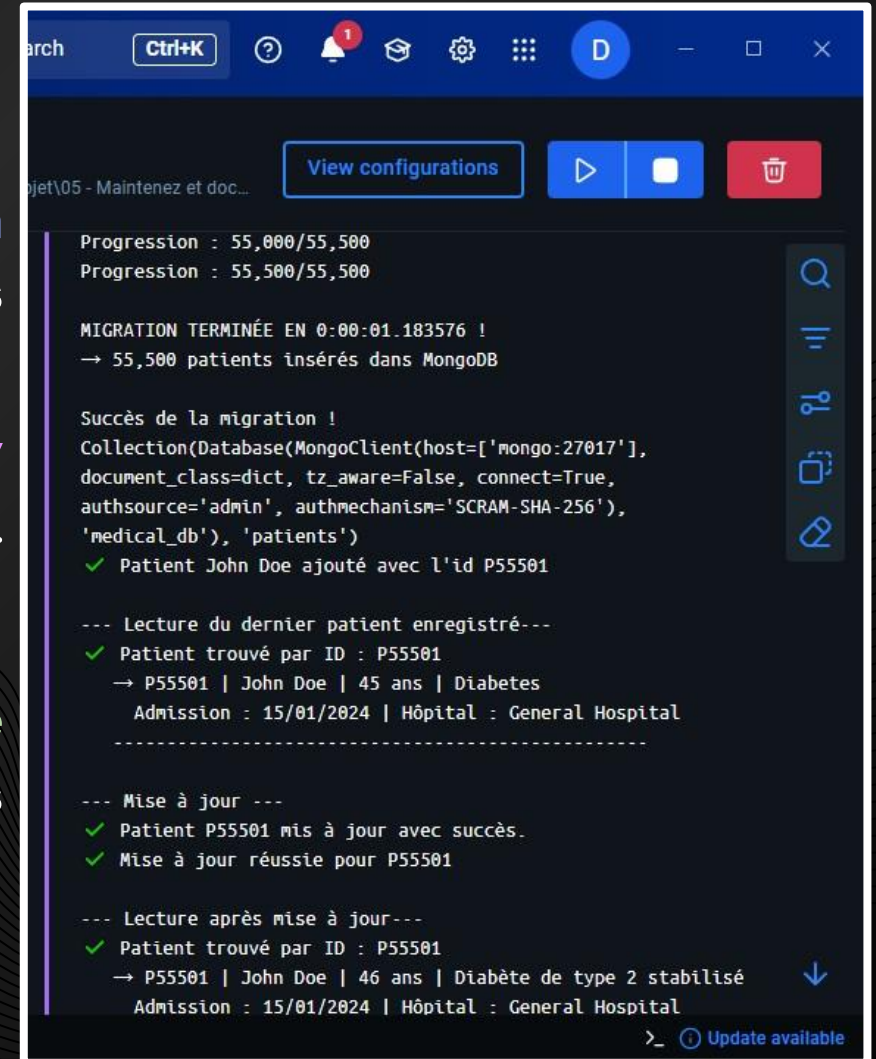
Tests et Vérification

Connexion mongosh
Vérification users, collections, données

Exécution main.py
Migration auto + Chaîne de tests CRUD + Exports.

Persistence prouvée
down/up sans perte de données

Docker desktop



```
Progression : 55,000/55,500
Progression : 55,500/55,500

MIGRATION TERMINÉE EN 0:00:01.183576 !
→ 55,500 patients insérés dans MongoDB

Succès de la migration !
Collection(Database(MongoClient(host=['mongo:27017'],
document_class=dict, tz_aware=False, connect=True,
authsource='admin', authmechanism='SCRAM-SHA-256'),
'medical_db'), 'patients')
✓ Patient John Doe ajouté avec l'id P55501

--- Lecture du dernier patient enregistré---
✓ Patient trouvé par ID : P55501
→ P55501 | John Doe | 45 ans | Diabetes
Admission : 15/01/2024 | Hôpital : General Hospital
-----

--- Mise à jour ---
✓ Patient P55501 mis à jour avec succès.
✓ Mise à jour réussie pour P55501

--- Lecture après mise à jour---
✓ Patient trouvé par ID : P55501
→ P55501 | John Doe | 46 ans | Diabète de type 2 stabilisé
Admission : 15/01/2024 | Hôpital : General Hospital

>_ Update available
```

Recherches AWS

Document créé dans PDF/AWS.pdf



Déploiement sur AWS

Amazon DocumentDB : Service géré compatible MongoDB (scalabilité, backups auto)

Amazon ECS/Fargate : Déploiement conteneurs Docker (notre app locale directement portable)



Avantages AWS

Création compte : Compte gratuit disponible

Tarification : Pay-as-you-go (économique pour scalabilité)

Sauvegardes : Automatiques + Point-in-time recovery

Surveillance : CloudWatch + Monitoring intégré

Sécurité : VPC, IAM roles, encryption

Conclusion

- 🚩 La migration locale est complète, sécurisée et reproductible grâce à Docker, avec authentification par rôles et persistance des données.

La solution est prête pour un déploiement AWS (via Amazon DocumentDB ou ECS/Fargate).

Bénéfices pour le client :

- Scalabilité horizontale pour gérer la croissance des données.
- Sécurité renforcée des données médicales sensibles.
- Exports facilités (JSON, CSV, Excel).